quy gán nguồn cho rag hai tầng

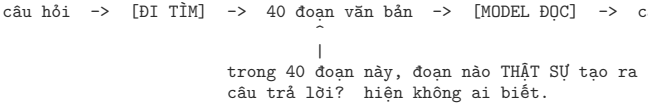
tìm *đoạn văn bản nào* đã tạo ra câu trả lời, và *vì sao* hệ thống tìm được chúng

tầng a — tìm **tập đoạn nhỏ nhất đủ để trả lời**
lý thuyết trò chơi hợp tác

tầng b — tìm **đường đi** trên đồ thị tri thức
integrated gradients

rag là gì, và nó thiếu gì

hỏi một model ngôn ngữ về sự kiện có thật thì nó hay *bịa*. cách chữa phổ biến tên là **rag**: trước khi trả lời, đi tìm vài đoạn văn bản liên quan rồi đưa kèm câu hỏi cho model đọc.



vì sao «không ai biết» là vấn đề

câu trả lời không kèm nguồn là câu trả lời **không dùng được** trong y tế, pháp lý, tài chính — người đọc không kiểm lại được.

và nó cũng chặn việc cải thiện hệ thống

không biết đoạn nào đúng thì **không dạy** được bộ đi tìm lần sau tìm tốt hơn — không có nhãn để học.

ví dụ chạy suốt buổi

```
câu hỏi   ai đạo diễn bộ phim thắng giải oscar phim hay nhất năm 1994?
đáp án    Robert Zemeckis

hệ thống lấy về 40 đoạn.  ba đoạn đáng chú ý:

  đoạn  3   "Forrest Gump thắng oscar phim hay nhất năm 1994."
  đoạn 12   "Robert Zemeckis đạo diễn Forrest Gump."
  đoạn 19   "Forrest Gump là hiện tượng văn hóa của thập niên 1990."

  ... và 37 đoạn khác, phần lớn không liên quan
```

đáp số mà tầng a phải tự tìm ra

đúng hai đoạn: 3 và 12.

đoạn 3 cho biết *phim nào*, đoạn 12 cho biết *ai đạo diễn*. **không đoạn nào một mình** trả lời được — phải **bắc cầu** qua hai đoạn.

đoạn 19 là cái bẫy của cả bài

đoạn 19 **không** nói ai đạo diễn, cũng **không** nói phim thắng giải.

nhưng nó *nhắc tên phim* — và chỉ cần thế là model nhớ ra đáp án từ kiến thức sẵn có rồi trả lời đúng. ta sẽ phải loại nó.

hai câu hỏi, hai công cụ

«đoạn nào» và «vì sao tìm ra» là hai bài toán khác hẳn nhau, nên dự án tách làm hai tầng.

tầng a — đoạn nào gánh câu trả lời

trong 40 đoạn đã lấy về, tập nào *thật sự* đủ để trả lời, và không đoạn nào thừa?

ở ví dụ trên, đáp số là $\{3, 12\}$

tầng b — vì sao bộ đi tìm tìm ra chúng

hệ thống đi tìm bằng cách lần theo một *đồ thị tri thức*. *đường* nào đã dẫn nó tới đoạn 3 và đoạn 12?

ví dụ: câu hỏi → «Forrest Gump» → «Zemeckis» → đoạn 12

hai tầng nối nhau bằng **một thứ duy nhất**: tầng a trả về tập đoạn T, tầng b nhận T rồi đi tìm đường. **không điểm số nào đi qua ranh giới** — lý do ở cuối buổi.

một truy vấn chạy hết pipeline

```
        3 nguồn tìm kiếm  ->  pool C(q) = 40 đoạn ứng viên
          |
TẦNG A  c0   chẻ câu hỏi thành các MẢNH nhỏ, rồi lập BẢNG PHỦ          1 lượt
          |      mảnh nào không đoạn nào nói ra  ->  BỎ truy vấn        (~45%)
        c1   đo v(rỗng)   model trả lời mà không được đọc gì            1 lượt
        c3   đo v(R)      cho đọc 12 đoạn tốt nhất                      1 lượt
        c4   đo v(C)      cho đọc cả 40 đoạn                            1 lượt
          |      ->  từ ba số này chốt NGƯỠNG tau_q của riêng câu hỏi q
        c2   g_theta - mô hình rẻ, đoán điểm thay cho llm               0 lượt
        c5   ứng viên: tập đọc từ trích dẫn ở c0, rồi tập phủ đủ mảnh   0 lượt
        c6   đo điểm THẬT bằng llm trên ứng viên và các tập con của nó ~6 lượt
        c7   đủ & nhỏ nhất -> XUẤT  |  chưa đủ -> thử tiếp  |  chạy lại đầy đủ
           v
        T = {đoạn 3, đoạn 12}              <-  THỨ DUY NHẤT qua ranh giới
          |
TẦNG B  b1   chấm điểm từng cạnh đồ thị, ở từng bước truyền tin
        b2   nối các cạnh điểm cao thành một ĐƯỜNG liền mạch
        b3   kiểm lại đường đó bằng model thật
           v
        đường đi  +  độ tin cậy  +  credit chia cho từng đoạn
```

40 đoạn ứng viên ở đâu ra

trước khi làm gì khác, phải có một rổ đoạn để chọn. rổ này gọi là **pool**, ký hiệu $C(q)$ — «q» là câu hỏi.

ba nguồn, mỗi nguồn một hạn ngạch

```
bm25          20 đoạn   khớp TỪ KHOÁ
gfm-rag       15 đoạn   đi theo ĐỒ THỊ
ngẫu nhiên     5 đoạn   lấy bừa trong kho
                       ----
                         40 đoạn = pool C(q)
```

thô tối đa 60, khử trùng lặp, rồi xếp hạng lại *trong từng nguồn* — không trộn chung, không thì nguồn mạnh nuốt hết chỗ

vì sao phải có nguồn ngẫu nhiên

nếu pool chỉ gồm đoạn mà bộ đi tìm *cho là* đúng, thì nhãn ta đào ra chỉ xác nhận lại thành kiến của chính nó.

5 đoạn bừa là **đối chứng**: nếu chúng cũng hay lọt vào đáp số thì phép đo của ta đang hỏng.

cả bài toán từ đây là: trong 2^{40} tập con của 40 đoạn này, **chỉ ra đúng một tập**. không thể thử hết — 2^{40} là hơn nghìn tỉ.

c0 · chẻ câu hỏi thành mảnh

chẻ **câu hỏi *và* đáp án vàng** thành những mẩu sự thật nhỏ — ta gọi là **mảnh**. được đọc *y* vì tầng a chạy *offline để đào nhãn*.

vào là *q* *và* *y* — ra là *J* mệnh đề

```
vào   q  ai đạo diễn phim thắng oscar 1994?
      y  Robert Zemeckis

ra    J = 2 mảnh
  mảnh 1  phim thắng oscar 1994 là
          Forrest Gump
  mảnh 2  Forrest Gump do Zemeckis
          đạo diễn
```

chỗ dễ hỏng: thực thể bắc cầu

«Forrest Gump» không nằm trong *q* lẫn *y*. bắt llm *nhớ* thì đoán sai ⇒ mảnh vô nghĩa ⇒ truy vấn **thoát ở c0 như thể pool thiếu**.

chữa: cho llm *đọc* thay vì *nhớ*

«Forrest Gump» nằm sẵn trong **đoạn 3 của pool** — nên đưa pool vào prompt, và bắt mỗi mảnh kèm **trích nguyên văn**.

trích không có trong pool = llm đang bịa, loại bằng phép so chuỗi. vẫn **1 lượt mỗi truy vấn**

c0 · đối chiếu từng đoạn với từng mảnh

với mỗi cặp (đoạn k, mảnh j), hỏi một model nhỏ **chạy trên máy mình**: *«đọc đoạn này có suy ra được câu kia không?»*

công cụ: một model nli nhỏ

```
premise     nội dung đoạn k      ~100 chữ
hypothesis  mảnh j               1 câu
                 |
                 v
        entail / neutral / contradict
```

model ∼400M tham số, **0 lượt gọi api**, $40 \times J$ lượt chạy — vài giây trên máy mình

nên mảnh PHẢI là câu khẳng định

nli chỉ được huấn luyện trên mệnh đề. đưa thẳng `"Who directed X? -> Y"` vào ô *hypothesis* là ngoài phân phối — **bảng phủ thành rác, mà không một cảnh báo nào**.

đây là lý do bước viết lại ở slide trước **không bỏ qua được**.

kết quả gọi là bảng phủ

ô (k, j) đánh dấu nếu đoạn k *nói ra* mảnh j — slide sau là bảng thật của ví dụ.

vì sao không dừng ngay ở đây?

trích dẫn đã chỉ thẳng mảnh nào ← đoạn nào. hợp lại là một tập ứng viên, **miễn phí**. vậy sao không trả luôn nó rồi xong?

và ta có dùng nó thật

```
mảnh 1 <- trích trong đoạn  3
mảnh 2 <- trích trong đoạn 12
                  |
                  v
        T_llm = {3, 12}     0 lượt thêm
```

đây thành **ứng viên đầu tiên** mà c5 thử — trước cả danh sách nghiệm phủ. đúng ngay thì phần tìm kiếm **bỏ qua được**.

nhưng nó là một *lời khai*, không phải một *phép đo*

1 chưa ai kiểm. cả thiết kế này đứng trên chỗ đó: $v(S)$ là *chạy thật* model trên S rồi đối đáp án. llm tự khai nó dùng đoạn nào là đúng loại việc **model vốn làm kém** — 35–62% ca nó trả lời đúng *khi ngữ cảnh không đủ*.

2 mỗi mảnh nó chỉ trích *một* đoạn. mà mảnh 1 còn được đoạn 7 đỡ nữa — nên ta **mất cả họ** W, mất luôn độ tin cậy và chỉ số cấu trúc.

3 không có gì nói tập con của nó không cũng đủ.

bảng phủ của ví dụ — 0 lượt llm

```
            mảnh 1   mảnh 2
đoạn  3     CÓ         -
đoạn  7     CÓ         -
đoạn 12       -        CÓ
đoạn 23       -        CÓ
đoạn 31       -        CÓ
đoạn 19       -         -     <- cái bẫy
... 34 đoạn còn lại đều trống
```

đoạn 19 trống cả hai cột — nó không nói ra mẩu sự thật nào. bảng phủ loại nó mà **không tốn một lượt llm nào**.

viết lại cho gọn

với một tập đoạn S, gọi $\mathrm{cov}(S)$ là tập các mảnh mà S nói ra được:

$$\mathrm{cov}(S) = \{\, j : \exists k \in S,\ (k, j) = \text{CÓ} \,\}$$

$$\mathrm{adq}(S) \iff \mathrm{cov}(S) = \{1, \ldots, J\}$$

adq đọc là «*adequate*» — S **phủ hết** mọi mảnh

ví dụ: $\mathrm{cov}(\{3\}) = \{1\}$ nên $\mathrm{adq}(\{3\})$ sai · $\mathrm{cov}(\{3, 12\}) = \{1, 2\}$ nên $\mathrm{adq}(\{3, 12\})$ **đúng**

c0 · vì sao phải chẻ ra, thay vì hỏi thẳng

câu hỏi tự nhiên: sao không đưa thẳng cả tập S và cả câu hỏi cho model nli? hai lý do, độc lập nhau.

lý do 1 · độ dài

cả tập S có thể dài tới $10{,}000$ chữ, mà model nli chỉ đọc được 512.

chẻ ra thì mỗi lần kiểm chỉ là **1 đoạn** ($\sim$100 chữ) đối **1 câu ngắn**. vừa khít.

lý do 2 · bắc cầu

hỏi thẳng *«đoạn này có trả lời được câu hỏi không?»* thì đoạn 3 trả lời **không**, đoạn 12 cũng **không**.

cả hai đều đúng — và ta **loại oan** $\{3, 12\}$, tức loại đúng loại câu hỏi nhiều chặng mà cả dự án nhắm tới.

chẻ ra thì mỗi mảnh có *đúng một* đoạn khẳng định nó, ghép lại mới thành câu trả lời. đó chính là hình dạng của một câu hỏi nhiều chặng.

bảng phủ dựng xong rồi thì ba thứ sau có ngay, không tốn thêm gì.

1 · cửa thoát, và sức khoẻ của mảnh

```
mảnh MỒ CÔI   không đoạn nào phủ
              -> pool thiếu, hoặc mảnh bìa
mảnh RỖNG     quá nửa pool phủ được
              -> quá chung chung, adq
                 thành LUÔN ĐÚNG
```

mồ côi thì thoát ngay, **trước khi tiêu một lượt llm nào**. rỗng thì **âm thầm hơn**: không ai thoát, bộ lọc chỉ lặng lẽ ngừng lọc.

3 · đoán trước cỡ đáp số

J mảnh thì đáp số *nên* có khoảng J đoạn. ví dụ: $J = 2$, và đáp số thật đúng là 2 đoạn.

2 · danh sách ứng viên, giải chính xác

```
mảnh 1 được phủ bởi:  {3, 7}
mảnh 2 được phủ bởi:  {12, 23, 31}

tập nhỏ nhất phủ hết cả hai mảnh:
  {3,12} {3,23} {3,31}
  {7,12} {7,23} {7,31}
                -> đúng 6 ứng viên
```

từ **hơn nghìn tỉ** tập con xuống **6 ứng viên**, bằng một phép tính tổ hợp chứ không phải phỏng đoán.

bài toán này tên là *set cover*; ở cỡ $J \leq 4$ thì giải vét cạn trong mili-giây

đo «trả lời đúng» — con số $v(S)$

$v(S)$ là điểm khi *chỉ* cho model đọc tập S rồi bắt trả lời. càng gần 1 càng đúng. mỗi lần đo tốn **hai lượt gọi llm**.

lượt 1 · bắt model trả lời, chỉ từ S

```
Answer using ONLY the passages below.
If insufficient, say: INSUFFICIENT.

[1] <nội dung đoạn 3>
[2] <nội dung đoạn 12>
Question: ai đạo diễn phim thắng oscar 1994?
Answer:
```

lượt 2 · một model khác chấm đúng/sai

```
Gold answer: Robert Zemeckis
Also acceptable: Zemeckis · Robert Lee Zemeckis
Candidate answer: <câu model vừa trả lời>
Correct? One word: Yes or No.
```

không lấy chữ «Yes», mà lấy *độ tin* của model vào chữ đó:

$$v(S) = \frac{\sum_{t\in\mathcal{Y}} e^{\ell_t}}{\sum_{t\in\mathcal{Y}} e^{\ell_t} + \sum_{t\in\mathcal{N}} e^{\ell_t}}$$

ℓ_t = điểm model gán cho từ t ở vị trí đầu · $\mathcal{Y}$ = {Yes, yes, ...}, $\mathcal{N}$ = {No, no, ...}

vì sao cần một số, không phải đúng/sai

đúng/sai cho 0 hoặc 1, nên hàng chục tập con **hòa nhau** và không so được tập nào hơn tập nào. có số thực thì mới xếp hạng được.

c1 c3 c4 · ba phép đo mốc, rồi chốt ngưỡng

$v(S) = 0{,}71$ là cao hay thấp? không trả lời được, vì mỗi câu hỏi một độ khó. nên phải đo *mốc* của riêng câu hỏi đó trước.

ba lượt đo, mỗi lượt một mốc

```
c1  v(rỗng) = 0,05   không cho đọc gì
                     -> model biết sẵn bao nhiêu
c3  v(R)    = 0,88   12 đoạn tốt nhất
c4  v(C)    = 0,81   cả 40 đoạn
```

trần $v_{\text{ref}} = \max($ ba số trên, và mọi $v(S)$ đo sau này $) = 0{,}88.$

$v(C) < v(R)$ *không* phải lỗi: nhồi 40 đoạn làm model phân tán. nên trần phải là **cực đại quan sát được**, không phải $v(C)$

ngưỡng đặt ở 95% quãng từ sàn lên trần:

$$\tau_q = \theta \left[v_{\text{ref}} - v(\emptyset) \right] + v(\emptyset), \qquad \theta = 0{,}95$$

```
tau_q = 0,95 (0,88 - 0,05) + 0,05 = 0,84
```

đọc ngưỡng này ra lời

«S đạt được **ít nhất** 95% phần mà *cả pool* làm được, tính từ mức model tự biết.»

chốt chặn: nếu $v_{\text{ref}} - v(\emptyset) < 0{,}05$ thì model vốn đã tự trả lời được, truy hồi chẳng đóng góp gì — trả $T = \emptyset$ và gắn cờ. ca này có thật, không phải lỗi.

ngưỡng tăng dần mà không tự mâu thuẫn

trần là «cực đại quan sát được», nên đo thêm thì trần có thể tăng, kéo ngưỡng tăng theo. vậy tập vừa nâng trần có bị chính ngưỡng mới loại không?

$$v(S) > v_{\text{ref}} \implies v_{\text{ref}} := v(S) \implies \tau_q' = \theta\big[v(S) - v(\emptyset)\big] + v(\emptyset) \underbrace{<}_{\text{vì } \theta < 1} v(S)$$

không, và $\theta < 1$ là lý do duy nhất

ngưỡng mới luôn **thấp hơn** điểm của tập vừa nâng trần. nên tập đó vẫn vượt ngưỡng. cập nhật không bao giờ loại chính cái gây ra nó.

và không phải xét lại quá khứ

τ_q chỉ *tăng*, nên tập nào đã trượt thì vẫn trượt — không cần chấm lại những tập đã bỏ.

nếu đặt $\theta = 1$ thì $\tau_q' = v(S)$ đúng bằng điểm của nó, và tính chất trên mất

giờ mới định nghĩa được «đủ» — và nó cần hai điều kiện

đã có $v(S)$ ở slide trước và $\mathrm{adq}(S)$ từ c0, nên nói được thế nào là *đủ*. điểm mấu chốt: **phải cả hai**.

$$S \text{ đủ} \iff \underbrace{v(S) \geq \tau_q}_{\text{model trả lời đúng}} \wedge \underbrace{\mathrm{adq}(S)}_{\text{các đoạn thật sự nói ra đáp án}}$$

nếu chỉ dùng v — quay lại đoạn 19

```
v({19}) = 0,71          cao
adq({19}) = SAI         phủ 0 / 2 mảnh
```

đoạn 19 chỉ nhắc *tên phim*; model tự nhớ ra đáp án rồi trả lời đúng.

bản chỉ-dùng-v sẽ **nhận đoạn 19 vào đáp số**, rồi đi dạy bộ truy hồi lần sau đi lấy đúng loại đoạn đó. hỏng từ gốc.

và lệch này đã được người khác đo

model trả lời đúng khi ngữ cảnh không đủ

trên các bộ đo trong bài	35–62 %

trong các ca model trả lời sai

do *không có* ngữ cảnh	10,2 %
do ngữ cảnh *có mà không đủ*	66,1 %

sau truy hồi, tỉ lệ ca thật sự đủ ngữ cảnh

musique-ans	46,2 %
hotpotqa	44,6 %

joren et al., iclr 2025 — arXiv:2411.06037

hai điều kiện cho bốn ô — và mỗi ô một nguyên nhân khác

giữ hai điều kiện *tách rời* thì mỗi ca hỏng rơi vào đúng một ô, và ô đó chỉ thẳng ra thủ phạm.

	adq **phủ hết mảnh**	adq **không phủ**
$v \geq \tau_q$	**nhãn tốt** — nhận	**mồi tham số** — loại, đây là đoạn 19
$v < \tau_q$	bằng chứng *có* mà model không nối lại được → lỗi **model sinh**, tầng b **không sửa được**	pool *thiếu* đoạn cần thiết → thoát sớm ngay ở c0

nếu gộp hai điều kiện thành một điểm số thì mất bảng này, và mất luôn khả năng tách *lỗi đi tìm* khỏi *lỗi model đọc*. ô dưới-trái là ô mà bản chỉ-dùng-v **không bao giờ nhìn thấy**.

«nhỏ nhất» nghĩa là gì — ba khái niệm từ lý thuyết trò chơi

coi mỗi đoạn là một *người chơi*, mỗi tập đoạn là một *liên minh*. v là tiền mà liên minh đó kiếm được. ngôn ngữ này đã có sẵn từ 1965.

ví dụ ba đoạn a, b, c — dùng lại ở cuối tầng a

```
tau_q = 0,84

v(rỗng)      0,05
v({a})       0,12
v({b})       0,18
v({c})       0,71   <- ĐƠN LẺ CAO NHẤT
v({a,b})     0,94   <- thắng
v({a,c})     0,60   <- thêm a lại GIẢM
v({b,c})     0,66
v({a,b,c})   0,80   <- THUA, vì có c
```

ba khái niệm, đọc thẳng từ bảng bên trái

thắng $v(S) \geq \tau_q$. ở đây chỉ $\{a, b\}$ và không ai khác

tối tiểu thắng, mà *bỏ bất kỳ đoạn nào* cũng thành thua

mwc tập thắng tối tiểu. $W = \{\{a, b\}\}$

v KHÔNG đơn điệu — thêm đoạn có thể làm tệ đi

$v(\{c\}) = 0{,}71$ mà $v(\{a, c\}) = 0{,}60$; và $\{a, b\}$ đang thắng thì thêm c làm nó **thua**. đoạn gây nhiễu làm model lạc thật, không phải nhiễu đo. ta **không sửa** — mwc vốn không cần đơn điệu.

bước c5 sắp tới phải so hàng nghìn tập con. gọi llm cho từng tập thì không trả nổi. nên huấn luyện g_θ — đoán $v(S)$, **0 lượt llm**.

$$g_\theta\Big(q,\ S,\ \{\hat{s}(q,k)\}_{k\in S},\ \{\tilde{v}_k\}_{k\in S},\ v(\emptyset)\Big)\ \longrightarrow\ \hat{v}(S)$$

$\hat{s}$ = điểm xếp hạng, luôn có sẵn · $\tilde{v}_k = v(\{k\}) - v(\emptyset)$ = đoạn k một mình đóng góp bao nhiêu · đảo thứ tự đoạn trong S không đổi kết quả

τ_q KHÔNG được làm đầu vào

g_θ đoán *điểm*; so với ngưỡng xảy ra sau, bên ngoài mô hình.

lý do thật: c2 chạy *trước* c4, mà c4 mới chốt τ_q — cho τ_q vào đầu vào là **tạo vòng tròn**. τ_q chỉ xuất hiện trong hàm mất mát lúc huấn luyện:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{rank}}$$

dữ liệu huấn luyện là nhật ký của chính các lượt đã gọi

pha 1, bốn vòng	1.110.000 nhãn
verification pha 2	1.880.000
tỉa khi đáp số to	104.000
ca phải chạy lại đầy đủ	821.000
ca bị bỏ	70.000
	3.983.000

không tốn thêm đồng nào: mỗi lần gọi llm để đo v, ta ghi lại cặp (tập con, điểm) — rồi dùng chính nó để dạy g_θ

dễ tự lừa ở đây. một con số đẹp vẫn có thể đi kèm một mô hình vô dụng.

cái bẫy: đo gộp mọi truy vấn

tập to thì v cao, tập nhỏ thì v thấp — đúng với *mọi* truy vấn. nên một mô hình **chỉ nhìn số đoạn** đã cho con số gộp rất đẹp.

mà việc ta cần là so các tập *trong cùng một* truy vấn, thường *cùng cỡ*. ở đó mô hình đó phân biệt được **bằng 0** — mọi tập cỡ 3 hòa điểm, và bước tìm thành chọn bừa.

nên đo thế này

auc tính trong từng truy vấn, rồi mới lấy trung bình qua các truy vấn.

sàn phải vượt: đúng cái mô hình chỉ nhìn $|S|$ ở trên. không vượt nó thì g_θ chưa học được gì.

và thứ tự thử kiến trúc

cây tăng cường (gbm) trên đặc trưng làm tay chạy **trước**. mô hình neural phức tạp chỉ đáng chọn nếu vượt được nó trên phép đo trên.

c5 · tìm đáp số — đi lên từ các tập phủ đủ mảnh

không duyệt 2^{40} tập. chỉ duyệt những tập *đã phủ hết mảnh*, xếp theo cỡ từ nhỏ lên — và danh sách đó c0 đã đưa sẵn.

các tầng

```
tầng 0   các tập phủ đủ, nhỏ nhất
         = 6 ứng viên c0 đã liệt kê
         {3,12} {3,23} {3,31} {7,12} ...
tầng 1   một tập phủ đủ  +  1 đoạn
tầng 2   một tập phủ đủ  +  2 đoạn
...

trong mỗi tầng:
   chấm toàn bộ bằng g_theta   (0 lượt)
   giữ lại vài ứng viên đầu bảng
   đo v THẬT theo đúng thứ tự đó
```

vì sao bỏ qua phần còn lại là an toàn

một tập *không* phủ hết mảnh mà vẫn vượt τ_q thì chỉ có thể vì **mỗi tham số** — mà ta đã quyết loại loại ca đó.

tìm ở đó là tìm đúng thứ mình sẽ vứt đi.

hai thứ tự nhiên đi kèm

1 mọi ứng viên đều đã đủ bằng chứng — không còn ca đoạn 19.

2 duyệt theo cỡ tăng dần, nên tập nhỏ luôn được thử trước: **«nhỏ nhất trước» là miễn phí**, không cần một bước tỉa riêng.

cách hiển nhiên là bỏ từng đoạn một rồi đo lại. cách đó **sai**, và lý do là *v* không đơn điệu.

một ca khác — ba đoạn x, y, z

```
tau_q = 0,76
v({x})     = 0,85  >= tau_q   <- x ĐÃ ĐỦ
v({x,y})   = 0,60  <  tau_q      y làm lạc
v({x,z})   = 0,71  <  tau_q
v({y,z})   = 0,40  <  tau_q
v({x,y,z}) = 0,80  >= tau_q

bỏ-một-đoạn từ {x,y,z}:
   ba cặp ĐỀU thua
   -> kết luận "tối tiểu"      SAI
sự thật:  {x} một mình đã thắng
   -> y và z đều thừa
```

muốn chắc thì phải quét *trọn* dàn tập con — $2^{|T|} - 2$ lượt. nhưng bảng phủ cắt phần lớn đi:

chỉ tập con CÒN PHỦ ĐỦ mới cần xét

```
T = {3, 12, 19}
  3 phủ mảnh 1 · 12 phủ mảnh 2 · 19 thừa

tập con thực sự:          7 tập
còn phủ đủ cả hai mảnh:   1 tập -- {3,12}
  {3,19}  mất mảnh 2
  {12,19} mất mảnh 1
  mọi tập một đoạn đều mất một mảnh
```

và c5 tầng 0 **đã đo** $\{3, 12\}$ **rồi** — nên chứng minh tối tiểu **tốn thêm 0 lượt**.

$\lvert T\rvert$	3	5	6	8	10
quét hết	\$0,02	\$0,11	\$0,24	\$1,14	\$5,37
tỉa trước rồi quét	—	\$0,11	\$0,04	\$0,07	\$0,10

c7 · ba nhánh kết thúc

sau c6 có đủ số liệu để quyết. ba khả năng, và chỉ khả năng thứ ba mới tốn tiền.

ba nhánh

```
(1) đủ, phủ hết mảnh, và tối tiểu
    -> XUẤT T sang tầng b                  xong

(2) chưa đủ  (v(T) < tau_q)
    -> thử ứng viên kế trong danh sách,
       tối đa 3 lần, mỗi lần CHỈ 1 lượt
    -> qua được: về nhánh (1)
       trượt cả 3: sang nhánh (3)

(3) đáp số quá to, hoặc tỉa bị kẹt
    -> CHẠY LẠI đầy đủ cho truy vấn này
```

nhánh 2 rẻ hơn tưởng rất nhiều

một lần thử tốn **1 lượt** ($1{,}250$ token); một lần chạy lại đầy đủ tốn $113{,}000$ — **90 lần thử mới bằng một lần chạy lại**. ba lần thử vớt được $\sim 66\%$ số ca trượt.

ca «đủ mà không tối tiểu» KHÔNG phải chạy lại

c6 đã đo *trọn* dàn tập con dưới T, nên tập nhỏ nhất đọc ra ngay — dùng lại đúng bảng x, y, z ở slide trước:

```
v({x,y,z}) = 0,80 >= tau_q    T đủ
   ba cặp con: đều thua
   v({x}) = 0,85 >= tau_q
              -> T KHÔNG tối tiểu
              -> T = {x}
```

ca này xảy ra thường xuyên, vì lỗi hay gặp nhất của g_θ là đề xuất tập **phình** ra quá cỡ

thường có nhiều đáp số đúng — chọn cái nào

ba đoạn cùng nói một ý thì có *ba* tập thắng tối tiểu, mỗi tập chứa một đoạn. hai thước đo sau để xếp hạng chúng.

$$\beta_i^+ = \big|\{\, S \text{ thiếu } i : v(S) < \tau_q \wedge v(S\cup\{i\}) \ge \tau_q \,\}\big| \qquad \beta_i = \frac{\beta_i^+ - \beta_i^-}{2^{\,n-1}} \qquad \mathrm{pgi}_i = \big|\{\, S \in W : i \in S \,\}\big|$$

β^+: thêm i biến *thua* thành *thắng* · β^-: thêm i làm *mất* chiến thắng · pgi: i có mặt trong bao nhiêu tập thắng tối tiểu

đếm tay trên đúng bảng a, b, c lúc nãy

```
i = a  {b} 0,18 -> {a,b} 0,94    ĐỔI
       ba ca kia: không           B+ = 1
i = b  {a} 0,12 -> {a,b} 0,94    ĐỔI
       ba ca kia: không           B+ = 1
i = c  không ca nào thua->thắng  B+ = 0
       {a,b} 0,94 thắng
         -> {a,b,c} 0,80 THUA    B- = 1
```

	a	b	c
$v(\{i\})$ một mình	0,12	0,18	**0,71**
β_i	0,25	0,25	$-0{,}25$
pgi_i	1	1	**0**

đây là cả lý do dùng lý thuyết trò chơi

c có *điểm đơn lẻ cao nhất* — xếp theo $v(\{i\})$ sẽ chọn nó. nhưng β_c **âm**: c chưa từng biến thua thành thắng, mà có lần *phá* mất một chiến thắng.

ba tiêu chí, lọc Pareto rồi gộp

```
càng nhỏ càng tốt     -|S|
cấu trúc              w_b*beta + w_p*pgi
hạng truy hồi         tổng r_i
                      (chỉ dùng phá hoà)
```

phải chuẩn hoá β và pgi riêng rồi mới cộng — $\beta \in [-1, 1]$ còn pgi đếm tới hàng chục, cộng thẳng thì pgi nuốt mất β

độ tin cậy, miễn phí

nếu chỉ *một* tập thắng rõ thì quy gán chắc. nếu năm tập suýt soát nhau thì **không chắc** — và ta báo ra con số đó thay vì giấu đi.

đầu ra của tầng a

```
T = S*              <- THỨ DUY NHẤT qua ranh giới

kèm theo, để chẩn đoán về sau:
+ phiên bản g_theta   lô nào gán nhãn
+ đường đi           thẳng · tỉa · thử lại · cứu
+ v(T) - tau_q       vượt ngưỡng bao nhiêu
+ cov(T) / J         phủ mấy trên mấy mảnh
+ W, entropy         mấy đáp số, chắc hay không
+ đoạn beta < 0      ứng viên đoạn gây hại
```

nhãn $(q \to T)$ không phụ thuộc kiến trúc model nào — đào một lần, dùng cho mọi lần huấn luyện sau.

tầng b — câu hỏi hoàn toàn khác

tầng a nói *đoạn 12 là nguồn*. tầng b hỏi: hệ thống đi *đường nào* để tới được đoạn 12? bộ đi tìm ở đây lần theo một đồ thị tri thức, truyền tin L bước từ các thực thể trong câu hỏi.

chấm điểm từng cạnh là KHÔNG đủ

điểm từng cạnh cho ra một *đống* cạnh quan trọng, không cho ra một *chuỗi*.

mà lời giải thích đọc được phải có dạng

```
q -> Forrest Gump -> Zemeckis -> đoạn 12
```

tức một **dãy cạnh nối tiếp**, mỗi bước ở đúng lượt truyền tin của nó.

ba bước — xương sống của cả phần này

```
b1 ĐO    chấm điểm từng (cạnh, bước)
         -> cạnh nào gánh, và gánh ở bước nào

b2 NỐI   trải phẳng đồ thị theo bước, rồi
         quy hoạch động tìm đường điểm cao nhất
         -> một dãy liền mạch, không phải đống

b3 THỬ   kiểm lại đường đó bằng model thật
         -> nó có thật sự đủ để giải thích không
```

đại lượng đem ra chấm

$$F_B \;=\; \log p(\text{đoạn } D)$$

xác suất model gán cho đúng đoạn đích. một lượt chạy cho ra xác suất của *mọi* đoạn, nên cả T dùng chung một lượt

integrated gradients — công cụ của bước b1

muốn biết một *núm* ảnh hưởng bao nhiêu, cách thô là vặn nó về 0 rồi xem kết quả tụt bao nhiêu. hàng nghìn núm thì phải chạy hàng nghìn lần.

ig làm khác

vặn *tất cả* núm từ 0 lên mức thật **cùng một lúc**, chia đường đi đó làm M chặng, mỗi chặng ghi lại độ nhạy của kết quả theo từng núm rồi cộng.

M lần chạy cho ra điểm của *mọi* núm.

$$\mathrm{ig}_i = (x_i - x_i^0) \times \frac{1}{M} \sum_{m=1}^{M} \frac{\partial F\big(x^0 + \alpha_m (x - x^0)\big)}{\partial x_i}$$

x^0 = mốc, mọi núm bằng 0 · $\alpha_m = m/M$ là chặng thứ m

tính chất đáng tiền: completeness

$$\sum_i \mathrm{ig}_i = F(x) - F(x^0)$$

tổng công của mọi núm cộng lại **đúng bằng** phần kết quả tăng lên — không dư, không thiếu. đây là một **đẳng thức kiểm được**, nên cài sai là biết ngay.

vì sao tầng a dùng cách thô, tầng b dùng ig

cạnh đồ thị	hàng nghìn, đồ thị CỐ ĐỊNH -> ig
đoạn / pool	prompt PHÌNH theo số đoạn
	-> không có điểm hoà vốn
	-> đành chạy thô

ta chèn vào mỗi cạnh một hệ số $g \in [0,1]$ — gọi là **gate**. để $g \equiv 1$ thì được đúng model gốc; hạ về 0 là bịt cạnh đó lại.

$$\text{bước truyền tin thứ } l: \qquad h_v^{(l)} \;=\; \mathrm{agg}\Big(\big\{\, g_e^{(l)} \cdot m_e^{(l)} \;:\; e=(u\to v) \,\big\}\Big)$$

một gate cho mỗi cạnh là KHÔNG đủ

cùng một cạnh có thể cần ở bước 1 mà vô dụng ở bước 3. một gate dùng chung **trộn hai vai** lại, và điểm trả về là một số vô nghĩa.

```
gate chung      g_e          |E| núm
gate theo bước  g_e^(l)      |E| x L núm
```

chỉ gate theo bước mới đọc ra được *thứ tự* — tức mới ra được một **chuỗi** thay vì một đống.

ig trên gate

$$\mathrm{ig}_e^{(l)} = g_e^{*(l)} \times \frac{1}{M}\sum_{m=1}^{M} \frac{\partial F_B\big(\alpha_m g^*\big)}{\partial g_e^{(l)}}$$

```
g* = 1      khi ĐÀO NHÃN
g* = ĝ      khi BÁO CÁO cho người đọc
```

$\hat{g}$ là hệ số model *tự* học được. dùng nó lúc báo cáo thì trung thực; nhưng dùng lúc đào nhãn thì sai — nó là thứ model *làm*, không phải thứ ta dùng để *đo*

một đường π gồm h cạnh và đóng góp c vào kết quả. completeness của ig nói thẳng rằng:

$$\sum_{e \in \pi} \mathrm{ig}_e = c \qquad \text{và nếu các cạnh trong } \pi \text{ đối xứng nhau thì} \qquad \mathrm{ig}_e = \frac{c}{h}$$

vì sao đây là thứ quý nhất của tầng b

cài sai bất cứ chỗ nào trong khâu ig — sai dấu, sai số chặng M, sai mốc, nhân hai lần hệ số — thì tổng **không còn bằng** c, và ta thấy ngay.

không có nó thì một lỗi cài đặt sẽ chạy êm, cho ra những đường trông hợp lý mà sai.

một đường, ba cạnh

```
c = 0,81            đóng góp của cả đường

ig cạnh 1 = 0,27
ig cạnh 2 = 0,27       mỗi cạnh = c/h
ig cạnh 3 = 0,27            = 0,81/3
            -----
tổng      = 0,81  = c         KHỚP
```

lệch $\Rightarrow$ có lỗi cài đặt. **dừng, đừng đi tiếp.**

b2 · nối các cạnh thành một đường

đồ thị gốc có chu trình nên không duyệt một lượt được. mẹo: *vẽ lại* nó — mỗi bước truyền tin một bản sao.

trải phẳng

```
node (v, l)  =  node v ở bước l
cạnh (u,l) -> (v,l+1)  nếu u->v trong G

L bước  ->  đồ thị KHÔNG CÒN CHU TRÌNH
        ->  duyệt được một lượt
```

thứ tự duyệt chính là số bước l — không cần sắp xếp gì thêm.

một lượt quét là xong

$$\text{best}(v,l) = \max_{u\,:\,(u,v)\in E} \Big[\, \text{best}(u,l-1) + \text{ig}^{(l)}_{(u,v)} \,\Big]$$

nhớ lại nơi đến từ đâu để truy ngược ra dãy cạnh · chi phí $O\big((|V|+|E|)\,L\big)$

cộng điểm, KHÔNG phải tìm đường ngắn nhất

quen tay thì đặt cost $= -\log(\cdot)$ rồi chạy dijkstra. **sai ở đây.**

```
ig CỘNG được, theo completeness:
   tổng ig trên một đường
   = đóng góp của đường đó

-log biến TỔNG thành TÍCH
   -> mất completeness
   -> c/h không còn đúng
   -> mất luôn phép kiểm nội bộ
```

và ig *có dấu* âm cho cạnh gây hại, mà $-\log$ của số âm thì vô nghĩa.

hai câu hỏi khác nhau, và hay bị lẫn: *tập đường đã lấy chiếm bao nhiêu phần bằng chứng* và *bỏ đúng đường này đi thì mất bao nhiêu*.

$$\mathrm{suff}(\Pi_D) \;=\; \frac{\sum_{(e,l)\,\in\,\Pi_D} \mathrm{ig}_e^{(l)}}{\sum_{(e,l)} \mathrm{ig}_e^{(l)}} \qquad\qquad \mathrm{nec}(\pi) \;=\; F_B(g^*) - F_B\big(g^*\ \text{bỏ}\ \pi\big)$$

suff tính trên TOÀN tập đường

không cộng suff của từng đường rồi lấy tổng — hai đường đi chung một cạnh thì cạnh đó bị đếm hai lần.

lấy **hợp các cạnh** rồi mới chia:

```
hợp cạnh: 0,22 + 0,19 + 0,33 = 0,74
          0,74 / 0,81 = 0,91
suff = 0,91       đủ
```

suff thấp có HAI nguyên nhân khác hẳn nhau

node nào không đi tới được từ bất kỳ điểm khởi nào vẫn nằm ở **mẫu số**. nên suff thấp chưa chắc do thiếu đường.

```
suff_max = trần mà mọi đường từ điểm khởi
           có thể đạt - đọc MIỄN PHÍ
           từ bảng quy hoạch động

suff_max thấp   -> nhận diện thực thể hỏng
                   dừng ngay, đừng tìm nữa
suff_max cao    -> đồ thị thiếu cạnh
```

b2 · khi cần nhiều hơn một đường

một đoạn có thể được chứng minh bởi vài đường hội tụ lại. gom dần cho tới khi đủ, rồi tỉa ngược những đường hoá ra thừa.

hai chặng

```
-- chặng 1: gom -------------------------
tập đường <- rỗng
lặp:
    lấy đường điểm cao nhất từ bảng qhđ
    gain = tổng ig các cạnh MỚI của nó
    thêm vào tập đường

    đủ (suff >= nguong)  -> DỪNG, thành công
    gain quá nhỏ         -> DỪNG, bằng chứng
                            phân tán
    quá n_max đường      -> DỪNG, hết ngân sách
                            người đọc chịu được

    đặt ig = 0 cho mọi cạnh vừa lấy
    chạy lại quy hoạch động

-- chặng 2: tỉa ngược -------------------
với từng đường, theo thứ tự đã thêm:
    bỏ nó ra mà vẫn đủ  ->  bỏ hẳn
```

cạnh dùng chung tự thành miễn phí

đặt ig $= 0$ cho cạnh đã lấy, nên vòng sau **không trả tiền lại** cho cạnh đó. hội tụ nhiều đường được xử lý *tự nhiên*, không cần luật riêng nào.

$n_{\max}$ không phải con số chọn bừa

$$n_{\max} = \left\lceil \frac{\theta_s}{\tilde{\varepsilon}} \right\rceil$$

mỗi đường được nhận phải góp ít nhất $\tilde{\varepsilon}$, nên quá $\theta_s/\tilde{\varepsilon}$ đường là không thể còn đủ. con số rơi ra từ hai ngưỡng đã có, không phải đặt tay

chỉ một thứ đi qua ranh giới

tầng a tính ra rất nhiều số. **không số nào** được đi sang tầng b. lý do không phải cho gọn — mà vì chúng *đo trên hai không gian khác nhau*.

giao diện

```
TẦNG A -> T = {đoạn 3, đoạn 12} -> TẦNG B

KHÔNG qua: v(S) · tau_q · beta · pgi
           g_theta · W · entropy
```

điểm của tầng a đo trên *rổ 40 đoạn*; tầng b đi trên *đồ thị*. mang một thang đo từ nơi này sang nơi kia là so hai thứ không so được.

T dùng làm nhãn để dạy bộ đi tìm

$$\mathcal{L}_{\text{node}} = \text{bce}\big(p(v),\ \mathbf{1}[v \in T]\big)$$

$$\mathcal{L}_{\text{hop}} = \sum_{l} \sum_{v \in V_{\text{hop}}(l)} \text{bce}\big(p^{(l)}(v),\ \mathbf{1}[(v,l) \in \Pi]\big)$$

$$\mathcal{L}_{\text{edge}} = \sum_{(e,l) \in \Pi} -\log \hat{g}_e^{(l)}$$

```
âm KHÓ = đúng node, SAI bước
```

không có loại âm khó này thì tỉ lệ dương:âm là $1 : 10^6$ và model học được cách luôn trả lời «không»

bảng chẩn đoán — và nó cũng là nguồn nhãn

hai trục độc lập: *đường có giải thích được không* và *bộ đi tìm có xếp đoạn đúng lên cao không*. bốn ô, bốn bệnh khác nhau.

	xếp hạng tốt	**xếp hạng kém**
suff **cao**	**lành** — tìm đúng, và giải thích được	**lỗi model** — đường có đó mà không xếp đoạn lên cao
suff **thấp**	**đường tắt** — xếp đúng *nhờ lối khác*, không nhờ bằng chứng	**lỗi đồ thị** — thiếu cạnh, hoặc nhận diện thực thể hỏng

ô *đường tắt* là ô dễ bỏ sót nhất: mọi chỉ số đều đẹp mà lời giải thích rỗng. và mỗi ô **tự nó là một lô nhãn** để huấn luyện vòng sau.

g_θ ở đâu ra — pha 1

cả timeline c0–c7 giả định đã có g_θ. nó được đào ở một pha riêng, chạy trước, trên một tập nhỏ hơn hai mươi lần.

pha 1 · 10,000 truy vấn · llm đầy đủ

```
a0  chẻ câu hỏi + bảng phủ   0 .. 2.150
a1  vòng 0  ba mốc + 40 đoạn
            đơn lẻ            44.500
a2  vòng 1  greedy biên       27.000
a3  vòng 2  học chủ động     ~37.000
a4  vòng 3  xác nhận lại     ~15.000
                            ---------
                            ~123.900 token
```

sản phẩm của pha 1 là g_θ, **không phải** nhãn cho tầng b.

pha 2 · 200,000 truy vấn · đường rẻ

```
c0  chẻ + bảng phủ
      musique có phân rã sẵn ->     0
      corpus khác, neo vào pool -> 2.150
      thiếu mảnh -> THOÁT SỚM
c1  v(rỗng)                        500
c2  g_theta chấm 40 đoạn đơn         0
c3  v(R)                         3.500
c4  v(C)                        10.500
      -> chốt tau_q
c5  tìm theo tầng bằng g_theta       0
c6  đo v thật, kiểm tối tiểu   ~7.400
c7  ba nhánh
                                --------
                                 ~22.300 token
```

rẻ hơn 5,5 lần. sản phẩm của pha 2 mới là nhãn ($q \to T$) cho tầng b. neo c0 vào pool thêm ∼$85 — đổi lấy việc llm *đọc* thay vì *nhớ* thực thể bắc cầu

năm cổng — và chúng nằm trước 75% chi phí

không cam kết ngân sách trước. chia thành các chặng, mỗi chặng kết thúc bằng một *cổng*: một phép đo rẻ quyết định có đi tiếp hay dừng.

	giai đoạn	chi phí	tích lũy	cổng chặn sau đó
0	hạ tầng · index test có sẵn	$0	$0	
0.5	chạy thử bốn cổng trên test index	$18	$18	**cổng 0.5** — chặn $1.100
1	hiệu chuẩn với chuẩn ngoài	$7	$25	**cổng 1** — adq có đáng tin?
2.0	dựng index cho phần pha 1	$74	$99	
2	pha 1 đào nhãn	$103	$202	
3	huấn luyện g_θ	$0	$202	**cổng 2** — vượt được sàn?
4	đo tỉ lệ p trên phần giữ lại	$0	$202	**cổng 3** — chặn $918
5.0	dựng index cho phần pha 2	$815	$1.017	
5	pha 2 + ca chạy lại	$103	$1.120	**cổng 4** $\times 10$ lô

tiêu trước **cổng 0.5** là $18; trước **cổng 3** là $202. thiết kế sai thì **không bao giờ phải trả $918 còn lại.**

model sinh chạy trên máy mình nên $0 tiền, nhưng ~ 53 giờ gpu — ràng buộc thật là **giờ gpu**, không phải tiền. $889 trong bảng là dựng index, vì index dựng sẵn chỉ có cho tập *test*

những gì chưa chắc — và cổng nào đo nó

năm con số dưới đây chưa ai đo. nói ra chúng, và nói luôn ai sẽ đo.

con số	trạng thái	ai đo
độ dài đoạn thật	giả định 250 token	`0.b` — sai thì **mọi** bảng chi phí sai, và **không cổng nào bắt được**
p — tỉ lệ qua được đường rẻ	giả định 0,75	cổng `0.5` đo rẻ, cổng `3` đo thật
45% thoát sớm ở c0	suy từ musique	`2.1` — chắc chắn *lạc quan*, vì ta cố ý dùng checkpoint chưa tinh chỉnh
adq sai bao nhiêu	chưa biết	cổng `1`, đối chiếu nhãn `supporting_documents`
cỡ rò rỉ nhãn vàng	chưa biết	`1.3` — chênh lệch giữa pool chỉ-bm25 và pool đầy đủ

hai cổng **miễn phí** phải chạy trước khi cam kết ngân sách: **cổng** p (500 truy vấn giữ lại, 0 lượt llm) và **chuẩn ngoài** (500 truy vấn test, $7). chúng rẻ *chính vì* nằm trước chi phí — bỏ qua để «chạy nhanh hơn» là bỏ đi toàn bộ giá trị của chúng.

tóm lại

ba quyết định thiết kế

1 «đủ» có *hai* thành phần. *v* đo *trả lời đúng*, adq đo *có bằng chứng*. chỉ dùng *v* thì nhận cả đoạn 19.

2 tìm *tập nhỏ nhất*, không xếp hạng điểm số. được cả *họ* lời giải thích và một thước đo độ tin cậy; cái giá là mất đẳng thức kiểm tra.

3 chỉ T đi qua ranh giới. hai tầng đo trên hai không gian khác nhau, nên điểm số không mang qua được.

hai thứ còn phải chứng minh

p — tỉ lệ truy vấn mà đường rẻ đi qua được. cả bảng chi phí treo vào nó, và nó **chưa được đo**.

adq — model nli loại đúng hay loại bừa. chưa biết, mà nó đang được giao quyền bỏ $\sim 45\%$ truy vấn.

bước tiếp theo

chạy `0.a`–`0.j` trên test index (đã dựng sẵn, \$0), rồi cổng `0.5` với \$18.

github.com/pqvu0307/minimal_set_rag-graph_learning